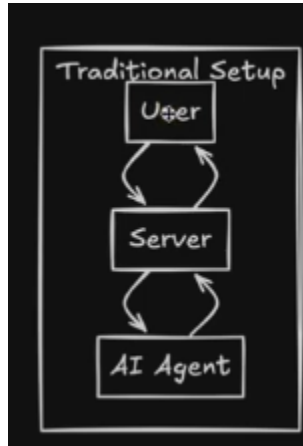


LiveKit and webRTC

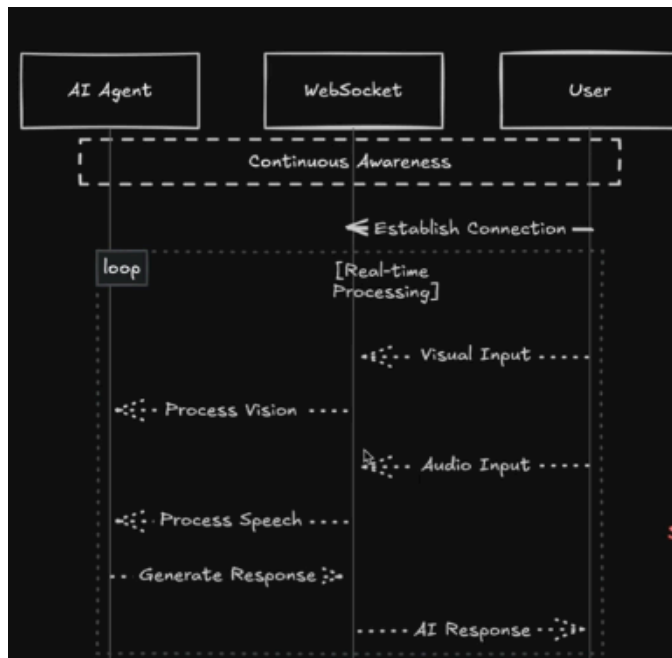
How does LiveKit solves the multimodal problem (webRTC101)

Http req-resp



Multiple requests with large data
Server overwhelm potential
Issues with RESTAPI(too many connection)

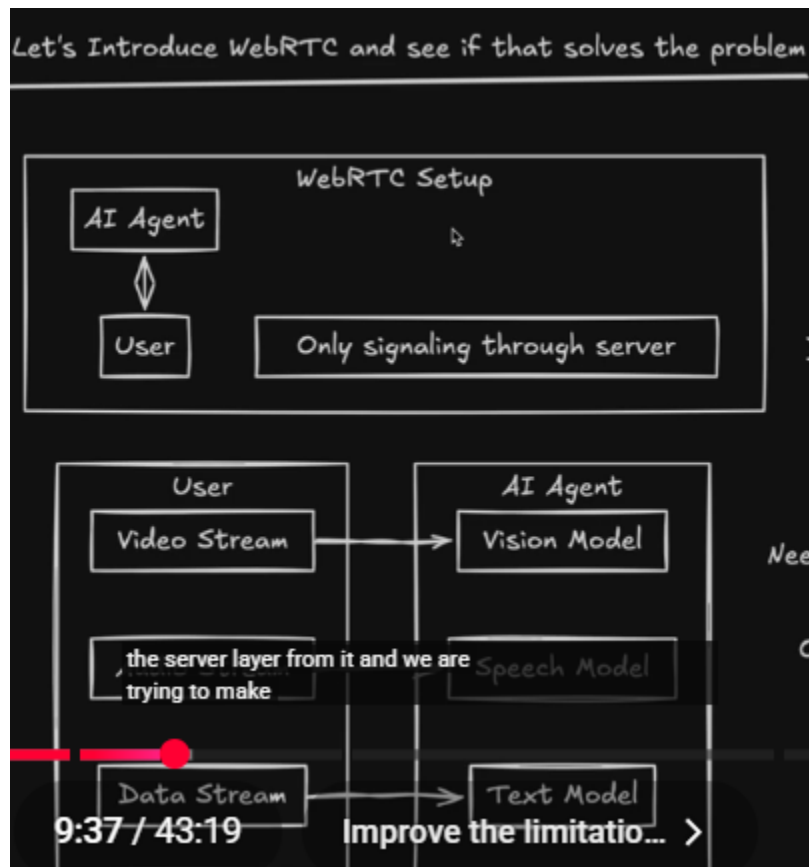
How websocket improve it



Enable bidirectional communication
Reduce connection overhead

Bandwidth constraints still exist
Network limitation affect performance

How webrtc improve it



Remove server bottleneck
P2p approach (user connect to user)

Need for initial discovery
(where is user, agent)
Signaling server (SDP) as
matchmaker
Once connected, direct p2p
communication

Problem: how to handle clients behind firewall or complex NAT setups?---

2) what happens with multiple client wanting to connect to same AI agent?

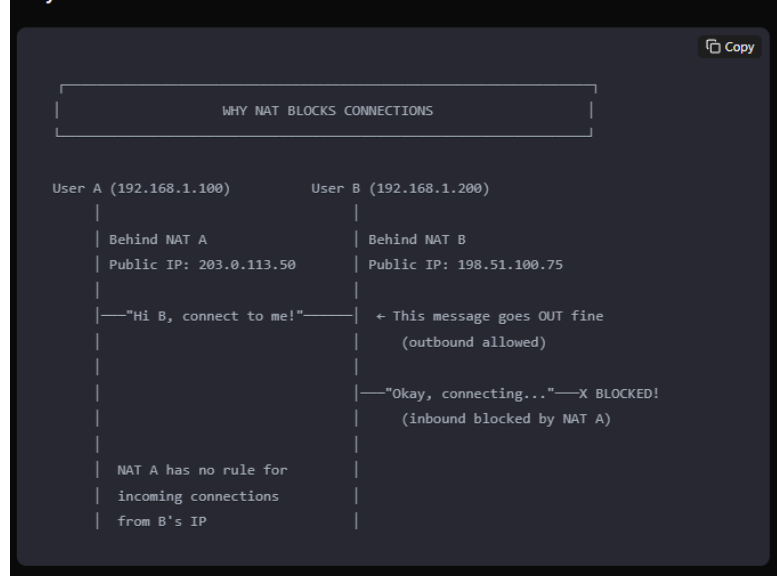
3) what about network and resource constraints on agent side?

What if there is client 1 and client 2 and agent 1 and agent 2. who would manage the state of these peer to peer communication and know who is busy and who is not

NAT & Firewall problem



Why NAT Blocks Direct Connections



Example:

Private IP: desk no. in office

Public ip: street no

Front desk: NAT

Building security: firewall

Case:

Cabin 201 send want to send a letter to desk 102 send to public ip street NAT(front desk) check the letter is for 102 desk and front desk give it to desk 102

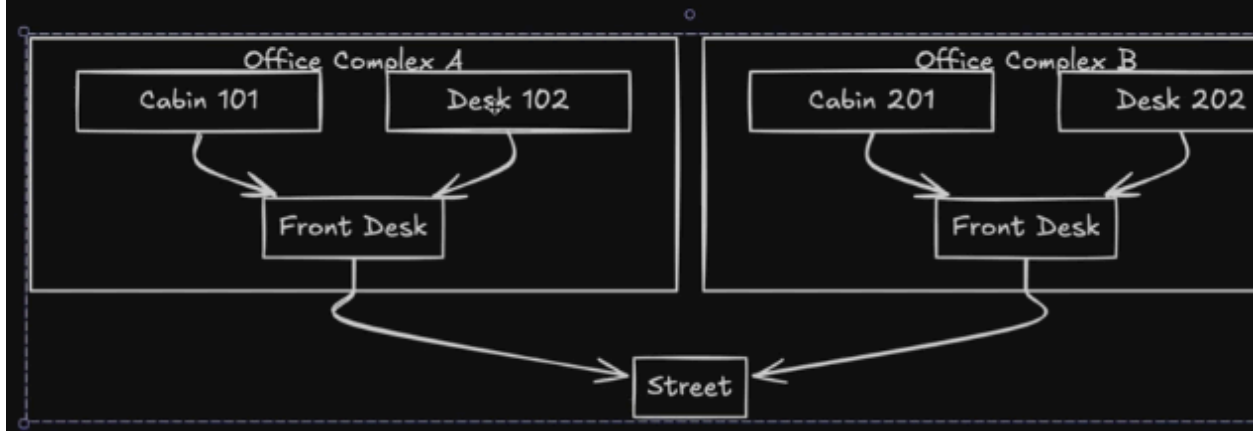
But in proper industry standard building 1 cabin 201 is not allowed to talk with building 2 desk 102 ,here firewall block it .p2p is not allowed.**TURN** will solve this

Understanding TURN and STUN Servers Traversal Using Relays around NAT Sess

Real-World Analogy

Imagine you live in a large apartment complex:

- Your Cabin/Desk number is your private IP (like 192.168.1.100)
- The building's street address is your public IP
- The front desk is like a NAT (Network Address Translation)
- Building security is like a firewall



Case2: but if cabin 201 want to talk with desk 202 .sturn will help it

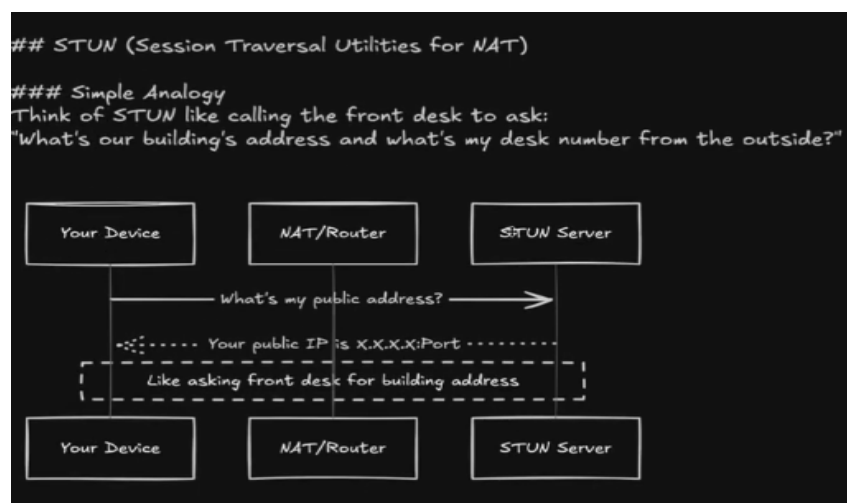
Problems:

When devices try to communicate:

- They don't know their public ip
- Nats block incoming connections
- firewall add additional restriction

STUN(Session traversal utilities for NAT)

STUN (Session Traversal Utilities for NAT) helps you discover your public IP address and port as seen from the internet.



How stun work

- Device sends req to stun server
- stun server sees devices public ip/port
- stun return this information to device
- Device can share this for direct connection

HOW TWO DEVICES DISCOVER EACH OTHER

BEFORE ANYTHING HAPPENS:

Device A	Device B
192.168.1.100 (private)	192.168.1.200 (private)
Behind NAT A	Behind NAT B
Public IP: ??? (doesn't know)	Public IP: ??? (doesn't know)

YOUR SIGNALLING SERVER (WebSocket)

|
| Both devices connected to this
|
└─ This is the MEETING POINT

STEP 1: BOTH DEVICES ASK STUN SERVER

Device A	STUN Server
"STUN server, what's	
my public IP?"	
→	
	Sees request from:
	203.0.113.50:54321
	(NAT A translated the address)
"Your public IP is	
203.0.113.50:54321"	
←	
Device A NOW KNOWS:	
"I am 203.0.113.50:54321	
from the internet's view"	
	Device B does the same:
	←
	"What's my public IP?"
	"Your public IP is
	198.51.100.75:12345"
	Device B NOW KNOWS:
	"I am 198.51.100.75:12345"



TURN(Traversal using Relay around NAT)

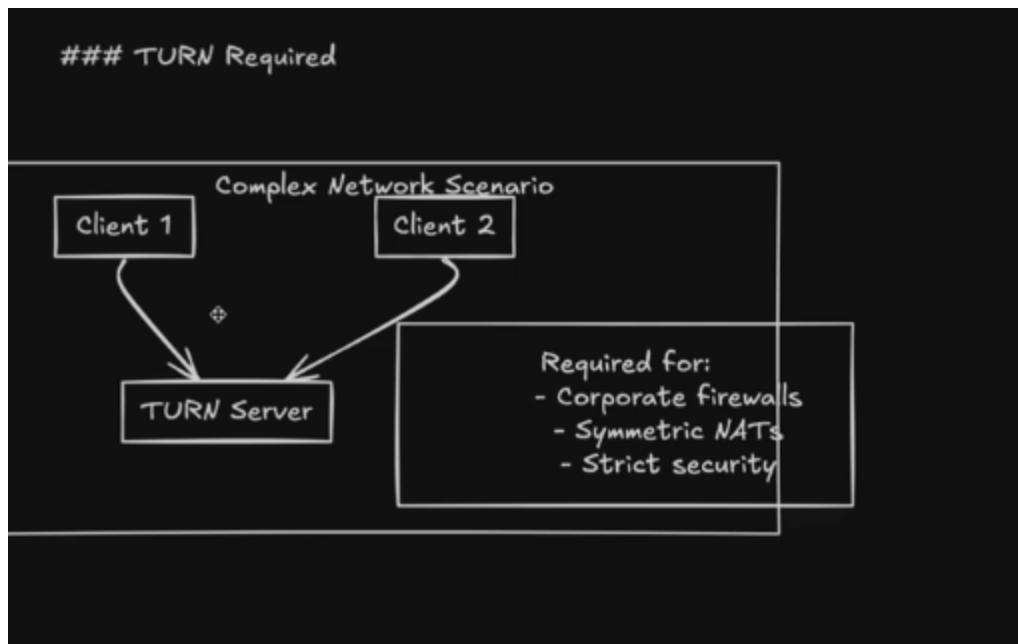
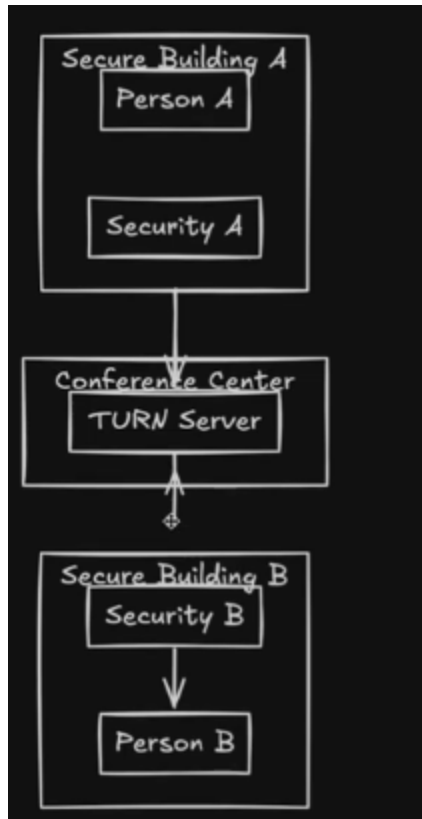
Imagine 2 people in different secure office building trying to meet:

-DIRECT MEETING(no turn needed):

Both can leave their building and meet in a public place

-TURN scenario:

- Building have strict security.
- they meet at a trusted conference center(TURN server)
- conference center relay messages between them



LIVEKIT

-Livekit provide it own turn server

LiveKit is an **open-source platform** that provides the building blocks for **real-time audio and video apps**. With support for **WebRTC, AI integrations**, and **flexible deployment options** (cloud or self-hosted), LiveKit helps developers build scalable, production-grade communication tools that meet the highest standards for performance and privacy.

LiveKit's architecture includes:

Client SDKs: Available for JavaScript, iOS (Swift), Android (Kotlin), Flutter, Unity, React Native, Node.js, Go, and Rust.

Room and Participant Management: APIs to create, manage, and moderate rooms and users.

Track-Level Controls: Fine-grained control to mute, pause, or adjust specific streams dynamically.

Webhooks and Events: For integration with third-party tools like analytics, moderation, or AI services.

Recording & Streaming: Capture sessions and broadcast them to platforms like YouTube, Twitch, or private servers.

Why Developers Choose LiveKit

Compared to other real-time video platforms, LiveKit offers:

Open Source Freedom: No lock-in, transparent development

Extensive SDK Support: Develop once, deploy anywhere

Customisable Infrastructure: Tailor the experience end-to-end

Self-Hosting Options: For privacy and control

Ready for AI: Integrate voice, vision, and NLP models natively

Active Developer Community: Ongoing improvements,
documentation, and support